

BASAVARAJESWARI GROUP OF INSTITUTIONS

BALLARI INSTITUTE OF TECHNOLOGY & MANAGEMENT



NACC Accredited Institution*
(Recognized by Govt. of Karnataka, approved by AICTE, New Delhi & Affiliated to
Visvesvaraya Technological University, Belagavi)
"JnanaGangotri" Campus, No.873/2, Ballari-Hospet Road, Allipur,
Ballari-583 104 (Karnataka) (India)
Ph: 08392 – 237100 / 237190, Fax: 08392 – 237197



DEPARTMENT OF CSE (DATA SCIENCE)

A Mini-Project Report On

“ONLINE LEARNING MANAGEMENT SYSTEM”

A report submitted in partial fulfillment of the requirements for the

NOT ONLY STRUCTURED LANGUAGE

Submitted By

PRIYANKA G

USN: 3BR23CD071

SANJANA V CHOUDHARI

USN: 3BR23CD086

SHIRISHA T S

USN: 3BR23CD088

T SRISHA

USN: 3BR23CD096

Under the Guidance of

Ms. B M Chaitra

Asst. Professor

Dept of CSE (DATA SCIENCE),

BITM, Ballari



Visvesvaraya Technological University

Belagavi, Karnataka 2025-2026

BASAVARAJESWARI GROUP OF INSTITUTIONS

**BALLARI INSTITUTE OF TECHNOLOGY &
MANAGEMENT**

NACC Accredited Institution*

(Recognized by Govt. of Karnataka, approved by AICTE, New Delhi & Affiliated to
Visvesvaraya Technological University, Belagavi)

"JnanaGangotri" Campus, No.873/2, Ballari-Hospet Road,

Allipur, Ballari-583 104 (Karnataka) (India)

Ph: 08392 – 237100 / 237190, Fax: 08392 – 237197



DEPARTMENT OF CSE (DATA SCIENCE)

CERTIFICATE

This is to certify that the OPEN-ENDED PROJECT of NOT ONLY STRUCTURED LANGUAGE title **“ONLINE LEARNING MANAGEMENT SYSTEM”** has been successfully presented by **PRIYANKA G** bearing USN **3BR23CD071**, **SANJANA V CHOUDHARI** bearing USN **3BR23CD086**, **SHIRISHA T S** bearing USN **3BR23CD088**, and **T SRISHA** bearing USN **3BR23CD096** students of semester B.E for the partial fulfillment of the requirements for the award of **Bachelor Degree in CSE(DS)** of the BALLARI INSTITUTE OF TECHNOLOGY & MANAGEMENT, BALLARI during the academic year 2024-2025.

Signature of guide

Ms. B M Chaitra

Signature of HOD

Dr. Aradhana D

ACKNOWLEDGEMENT

The satisfactions that accompany the successful completion of our mini project on **“ONLINE LEARNING MANAGEMENT SYSTEM”** would be incomplete without the mention of people who made it possible, whose noble gesture, affection, guidance, encouragement and support crowned my efforts with success. It is our privilege to express our gratitude and respect to all those who inspired us in the completion of our mini-project.

We are extremely grateful to our Guides **Ms. B M Chaitra** for their noble gesture, support co-ordination and valuable suggestions given in completing the mini-project. We also thank **Dr. Aradhana D**, H.O.D. Department of CSE(DS), for his co-ordination and valuable suggestions given in completing the mini-project. We also thank Principal, Management and non-teaching staff for their co-ordination and valuable suggestions given to us in completing the Open ended project.

<u>Name</u>	<u>USN</u>
PRIYANKA G	3BR23CD071
SANJANA V CHOUDHARI	3BR23CD086
SHIRISHA T S	3BR22CD088
T SRISHA	3BR23CD096

TABLE OF CONTENTS

Ch No	Chapter Name	Page
	Abstract	1
1	Introduction 1.1 About the project 1.2 Language used	2-3
2	Scope of the project 2.1 Problem statement 2.2 Objectives	4
3	System requirements 3.1 Functional Requirements 3.2 Non-Functional Requirements 3.3 Hardware Requirements 3.4 Software Requirements	5-7
4	Methodology 4.1 E R diagram 4.2 Schema Diagram	8-9
5	Implementation 5.1 Table Description 5.2 Source Code 5.3 Connections	10-15
6	Result	16-21
7	Conclusion	22
	References	23

ABSTRACT

This project presents the design and implementation of an Online Learning Management System (LMS) using MongoDB as a NoSQL database. The objective of the system is to efficiently manage courses, modules, videos, quizzes, student progress, and personalized recommendations. The platform enables students to access learning materials while instructors can monitor performance in real time.

The LMS backend uses MongoDB to store student profiles, course details, module content, quiz attempts, and progress analytics. Key insights such as course completion percentage, difficult modules (low quiz scores), inactive students, and leaderboard rankings are generated using MongoDB aggregation pipelines. The system demonstrates how NoSQL databases can support scalable EdTech applications by handling nested documents and unstructured learning data efficiently. This project provides a modern and effective solution for personalized learning, educational analytics, and real-time academic monitoring.

CHAPTER 1

INTRODUCTION

1.1 About the Project

The Online Learning Management System (LMS) with Progress Tracking is designed to support digital learning by enabling users to enroll in courses, watch video lectures, attempt quizzes, and track their performance. The system stores course structures, modules, videos, quiz attempts, and progress details using MongoDB.

The project focuses on providing:

- A structured course hierarchy
- Real-time progress tracking
- Personalized recommendations
- Analytics for student performance
- Identification of difficult modules
- Leaderboard based on quiz scores

MongoDB is chosen due to its flexible schema that easily handles nested learning data such as modules, videos, quiz questions, and student activities.

1.2 Language used

The project uses:

- **Python / Node.js** → Backend logic
- **MongoDB** → NoSQL database
- **JavaScript/HTML** (optional) → Presentation layer

MongoDB's document-oriented model is ideal for representing complex learning content structures.

CHAPTER 2

Scope of the Project

2.1 Problem Statement

Traditional relational LMS systems struggle with nested learning structures such as modules, videos, quizzes, attempts, and detailed progress tracking. This project aims to design a scalable MongoDB-based LMS that stores course content and student activity efficiently, while providing insights like progress percentage, weak areas, and student ranking.

2.2 Objectives

The primary objectives of the LMS are:

- Store student profiles and course details
- Maintain modules, videos, and quizzes
- Track student activity and progress
- Identify difficult modules through quiz analytics
- Provide personalized learning suggestions
- Monitor inactive students
- Generate leaderboards based on quiz scores

CHAPTER 3

System Requirements

3.1 Functional Requirements

User Management

- Add and manage student profiles
- Validate user information

Course Management

- Add and update courses
- Add modules and videos
- Store course hierarchy

Progress Tracking

- Track completed videos
- Calculate completion percentage
- Identify inactive students

Quiz Management

- Allow quiz attempts
- Store quiz scores
- Analyze difficult modules

Analytics

- Course completion analytics

- Student performance monitoring
- Leaderboard generation

3.2 Non-Functional Requirements

Performance

- Fast read/write operations
- Efficient aggregation for analytics

Security

- Secure access to student data
- Protection from NoSQL injection

Usability

- 4 Simple interface for students/instructors
- 5 Clear progress visualization

5.1 Hardware Requirements

- Server system for hosting MongoDB
- Development workstation
- Stable network infrastructure

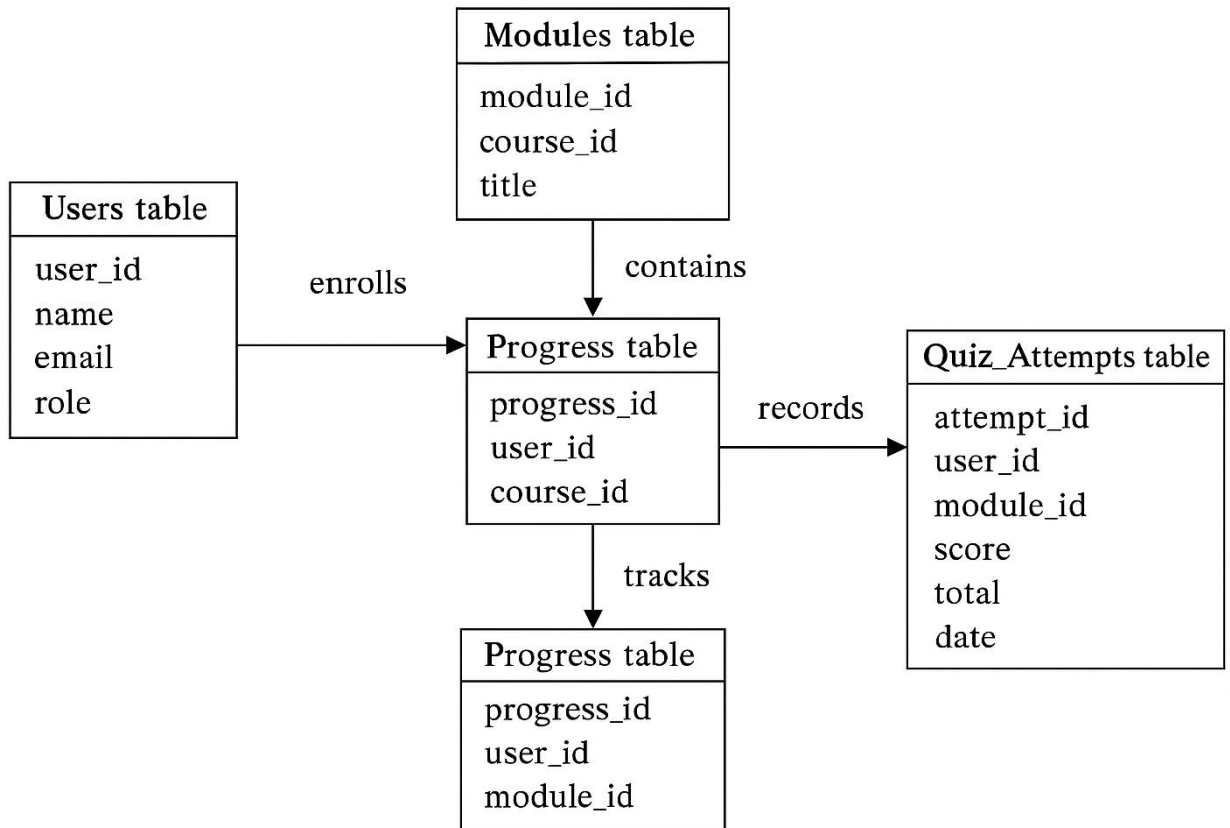
5.2 Software Requirements

- Operating System: Windows / Linux
- Database: MongoDB
- Backend: Python / Node.js
- Development Tools: VS Code, MongoDB Compass

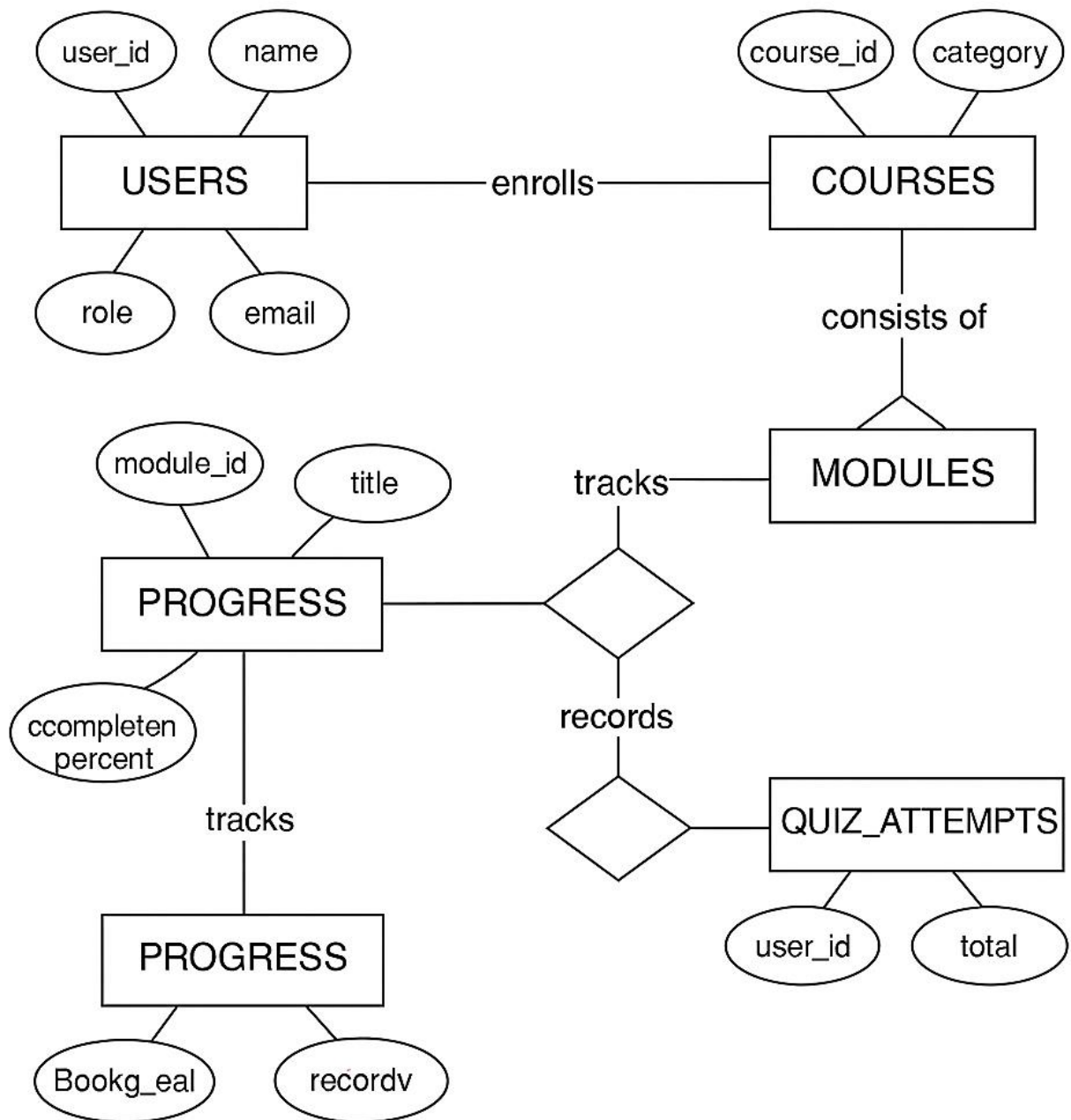
CHAPTER 4

Methodology

4.1 Schema Diagram



4.2 ER Diagram



CHAPTER 5

Implementation

5.1 Table Description

User Table:

- **user_id (INT AUTO_INCREMENT PRIMARY KEY):**
A unique identifier for each user. This is the primary key of the table.
- **name (VARCHAR(50)):**
The full name of the user.
- **email (VARCHAR(50)):**
The email address of the user used for communication and login.
- **role (VARCHAR(20)):**
Defines whether the user is a *student* or *instructor*.

Courses Table:

- **course_id (INT AUTO_INCREMENT PRIMARY KEY):**
A unique identifier for each course. This is the primary key.
 - **title (VARCHAR(100)):**
The title or name of the course.
 - **category (VARCHAR(50)):**
The subject area or domain of the course (e.g., Programming, Data Science).
-

Modules Table:

- **module_id (INT AUTO_INCREMENT PRIMARY KEY):**
A unique identifier for each module. This acts as the primary key.
- **course_id (INT):**
A foreign key referencing *course_id* in the Courses table.
- **title (VARCHAR(100)):**
The name of the module within the course.
- **videos (TEXT / JSON):**
A list of video lessons included in the module.

Progress Table:

- **progress_id (INT AUTO_INCREMENT PRIMARY KEY):**
A unique identifier for each progress entry. This is the primary key.
- **user_id (INT):**
A foreign key referencing the *user_id* in the Users table.
- **course_id (INT):**
A foreign key referencing the *course_id* in the Courses table.
- **completed_videos (TEXT / JSON):**
Stores the IDs or names of videos completed by the user.
- **completion_percent (INT):**
Shows the percentage of the course completed.
- **last_active (DATE):**
Indicates the date when the user last accessed the course.

Quiz_Attempts Table:

6 attempt_id (INT AUTO_INCREMENT PRIMARY KEY):

A unique identifier for each quiz attempt. This is the primary key.

7 user_id (INT):

Foreign key referring to *user_id* in the Users table.

8 module_id (INT):

Foreign key referring to *module_id* in the Modules table.

9 score (INT):

Marks obtained by the user in the quiz.

10 total (INT):

Total marks for the quiz.

11 date (DATE):

The date on which the quiz was attempted.

11.1 Source Code

```
from flask import Flask
from flask_cors import CORS
from config.config import Config
from config.db import mongo

app = Flask(__name__)
app.config["MONGO_URI"] = Config.MONGO_URI
app.config["SECRET_KEY"] = Config.SECRET_KEY

mongo.init_app(app)
CORS(app)

# Import blueprints AFTER initializing mongo
from routes.user_routes import user_bp
app.register_blueprint(user_bp, url_prefix="/api/users")

@app.route("/")
def home():
    return {"message": "LMS Backend Running"}

from flask import render_template, request, redirect

@app.route('/register', methods=['GET', 'POST'])
def register_page():
    message = None
    if request.method == 'POST':
        # Call your existing register logic here, or inline:
        data = request.form
        if mongo.db.users.find_one({"email": data["email"]}):
            message = "User with this email already exists"
        else:
            mongo.db.users.insert_one({
                "username": data["username"],
                "email": data["email"],
                "password_hash": data["password"] # In real code, hash password!
            })
            message = "User registered successfully!"
    return render_template('register.html', message=message)

from flask import render_template, request

@app.route('/login', methods=['GET', 'POST'])
```

```

def login_page():
    message = None
    success = False
    if request.method == 'POST':
        data = request.form
        user = mongo.db.users.find_one({"email": data["email"]})
        if user and user["password_hash"] == data["password"]: # Replace with
hash check for real apps!
            message = "Login successful!"
            success = True
            # You might set session, redirect, etc. here
            # In your login route, upon successful login:
            session['user_name'] = user.get('name', user.get('username', 'User'))
            session['user_id'] = str(user['_id']) # Store user_id in session
            #session['user_name'] = user['name'] # Store user's name

            return redirect(url_for('dashboard'))

    else:
        message = "Invalid email or password."
    return render_template('login.html', message=message, success=success)

```

```

from flask import render_template, session, redirect, url_for
from bson.objectid import ObjectId

```

```

from bson.objectid import ObjectId

```

```

@app.route('/dashboard')
def dashboard():
    user_id = session.get('user_id') # Always get user_id from session
    user_name = session.get('user_name', 'User')
    if not user_id:
        return redirect(url_for('login_page')) # Require login

    enrolled = mongo.db.enrollments.find({"user_id": user_id})
    enrolled_ids = [ObjectId(e["course_id"]) for e in enrolled]
    enrolled_courses = list(mongo.db.courses.find({"_id": {"$in":
enrolled_ids}}))
    other_courses = list(mongo.db.courses.find({"_id": {"$nin": enrolled_ids}}))
    return render_template("dashboard.html", user_name=user_name,
enrolled_courses=enrolled_courses, other_courses=other_courses)

```

```

from flask import render_template, session, redirect, url_for, request, flash

@app.route('/course/<course_id>', methods=['GET', 'POST'])
def course_view(course_id):
    user_id = session.get('user_id')
    course = mongo.db.courses.find_one({"_id": ObjectId(course_id)})
    # Check if already enrolled
    is_enrolled = mongo.db.enrollments.find_one({"user_id": user_id, "course_id":
str(course_id)})

    if request.method == 'POST' and not is_enrolled:
        mongo.db.enrollments.insert_one({"user_id": user_id, "course_id":
str(course_id)})
        flash("You have been enrolled in this course!", "success")
        return redirect(url_for('course_view', course_id=course_id))

    # Only show modules/progress/quizzes if enrolled
    modules, quizzes = [], []
    if is_enrolled:
        modules = list(mongo.db.modules.find({"course_id": course_id}))
        for m in modules:
            prog = mongo.db.progress.find_one({"user_id": user_id, "module_id":
m["_id"]})
            m["progress"] = prog["completion_percent"] if prog else 0
        quizzes = list(mongo.db.quizzes.find({"course_id": course_id}))

    return render_template(
        "course_view.html",
        course=course,
        enrolled=is_enrolled,
        modules=modules,
        quizzes=quizzes
    )

if __name__ == "__main__":
    app.run(debug=True, port=Config.PORT)

from flask import Blueprint
from controllers.user_controller import register_user, get_user
user_bp = Blueprint('user_bp', __name__)

user_bp.route("/register", methods=["POST"])(register_user)

```

```

user_bp.route("/<email>", methods=["GET"])(get_user)
#from app import
from config.db import mongo
from bson.objectid import ObjectId
from werkzeug.security import generate_password_hash, check_password_hash

class User:
    collection = mongo.db.users

    def __init__(self, username, email, password):
        self.username = username
        self.email = email
        self.password_hash = generate_password_hash(password)

    def save(self):
        return self.collection.insert_one({
            "username": self.username,
            "email": self.email,
            "password_hash": self.password_hash
        })

    @staticmethod
    def find_by_email(email):
        return User.collection.find_one({"email": email})

    @staticmethod
    def verify_password(stored_hash, password):
        return check_password_hash(stored_hash, password)

    @staticmethod
    def to_json(user_doc):
        return {
            "id": str(user_doc["_id"]),
            "username": user_doc["username"],
            "email": user_doc["email"]
        }
from models.user_model import User
from flask import request, jsonify

def register_user():
    data = request.get_json()
    if User.find_by_email(data["email"]):
        return jsonify({"message": "User with this email already exists"}), 400

    user = User(data["username"], data["email"], data["password"])

```

```
    user.save()
    return jsonify({"message": "User registered successfully!")), 201

def get_user(email):
    user_doc = User.find_by_email(email)
    if user_doc:
        return jsonify(User.to_json(user_doc)), 200
    return jsonify({"message": "User not found"}), 404
```

11.2 Connections

The database connection is established using the following parameters:

```
# config/db.py
```

```
from flask_pymongo import PyMongo
```

```
mongo = PyMongo()
```

Connection details:

- Host: localhost
- Database: hotel (created if not exists)

The connection is initiated at the start of the program and closed when the user chooses to exit

CHAPTER 6

Result

The screenshot displays two instances of the MongoDB Compass interface. The top instance shows the 'users' collection with three documents. The bottom instance shows the 'courses' collection with three documents.

Top Screenshot (users collection):

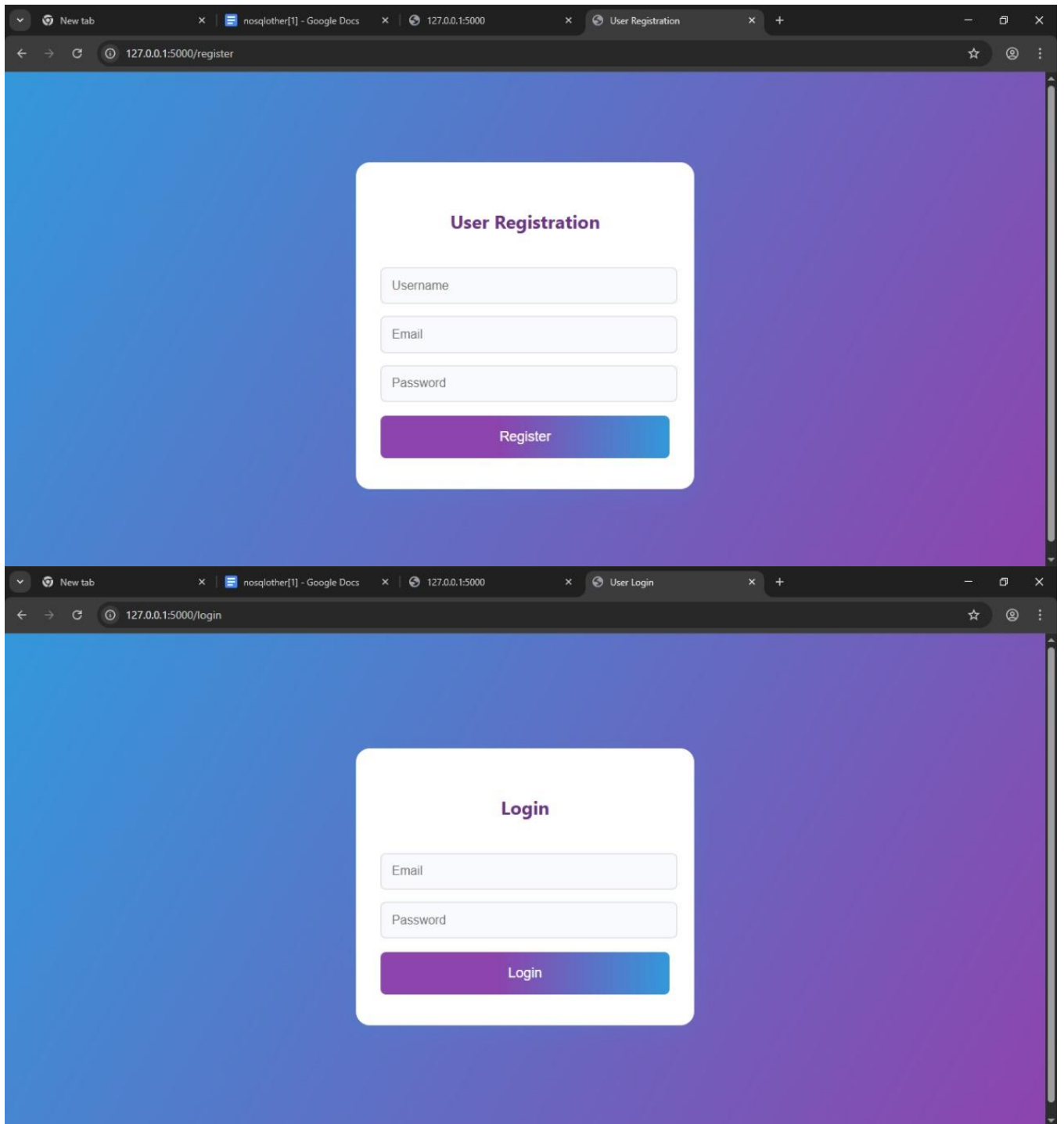
- Database: lms
- Collection: users
- Documents: 3
- Query: Type a query: { field: 'value' } or [Generate query](#)
- Results:

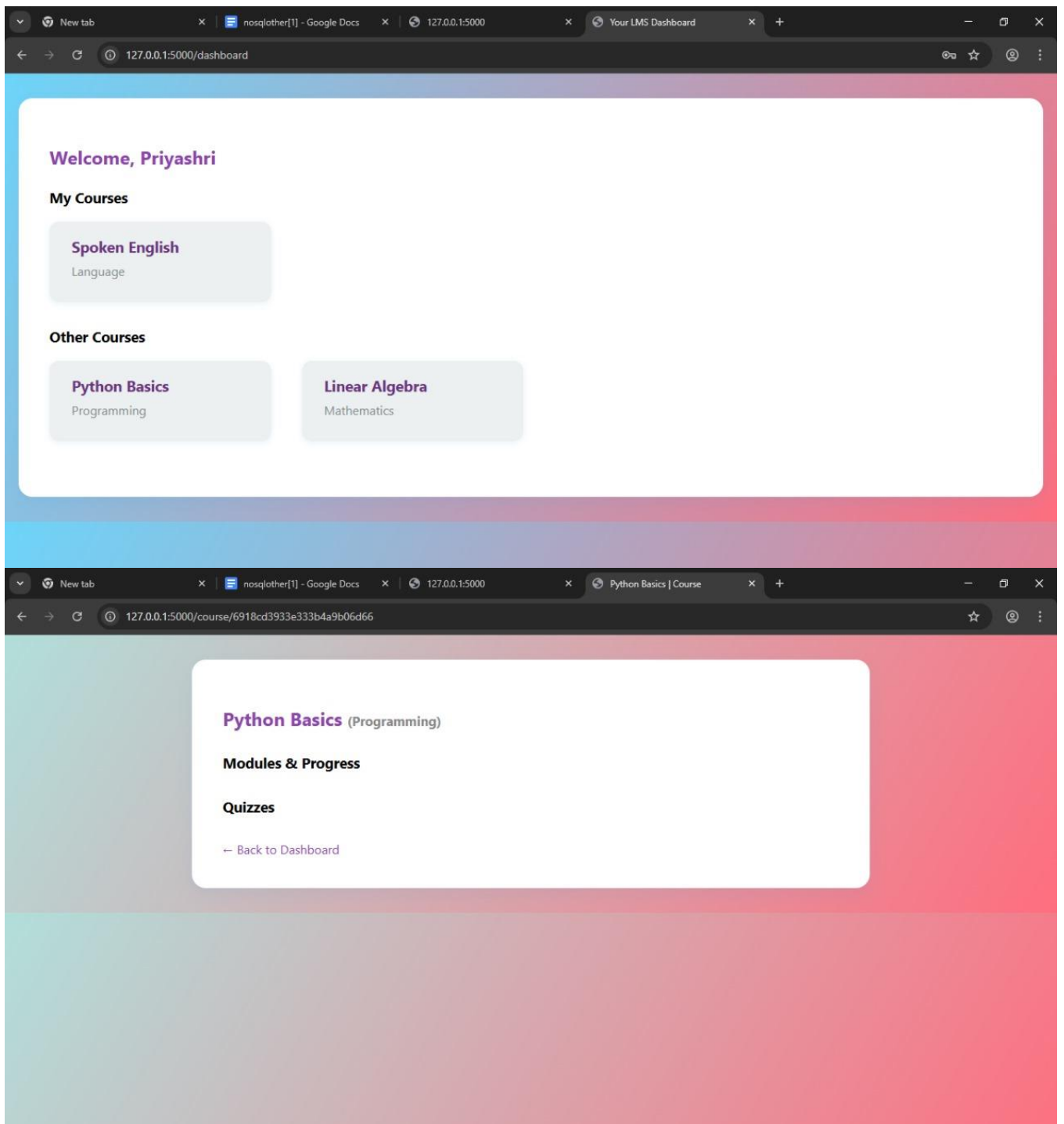
Document
<pre>{ "_id": ObjectId('6918b658be02f6ca7ed03cb2'), "username": "Sanjana Choudhari", "email": "soulbyte@gmail.com", "password_hash": "lms1" }</pre>
<pre>{ "_id": ObjectId('6918b7bfbe02f6ca7ed03cb3'), "username": "Soulbyte", "email": "abc@gmail.com", "password_hash": "lms2" }</pre>
<pre>{ "_id": ObjectId('69194bf67e10be5c6c3448d7'), "username": "Priyashri", "email": "siri.priya@gmail.com", "password_hash": "lms3" }</pre>

Bottom Screenshot (courses collection):

- Database: lms
- Collection: courses
- Documents: 3
- Query: Type a query: { field: 'value' } or [Generate query](#)
- Results:

Document
<pre>{ "_id": ObjectId('6918cd3933e333b4a9b06d66'), "title": "Python Basics", "category": "Programming" }</pre>
<pre>{ "_id": ObjectId('6918ce0b33e333b4a9b06d67'), "title": "Linear Algebra", "category": "Mathematics" }</pre>
<pre>{ "_id": ObjectId('6918ce0b33e333b4a9b06d68'), "title": "Spoken English", "category": "Language" }</pre>





CHAPTER 7

Conclusion

The Online Learning Management System (LMS) using MongoDB provides an efficient solution for managing digital learning content, tracking student progress, and performing academic analytics. Using MongoDB's document-oriented database helps store complex nested structures such as modules, videos, quiz attempts, and activity logs effectively.

The system successfully generates course analytics, identifies weak areas, flags inactive students, and supports personalized recommendations. With scalable architecture, flexible schema, and real-time performance insights, the project serves as a modern EdTech backend model and demonstrates the advantages of NoSQL databases in academic platforms.

References

- [1] K. Chodorow, *MongoDB: The Definitive Guide*. O'Reilly Media, 2019.
- [2] D. Barry and T. Stanienda, "NoSQL Databases and Their Application in Big Data Analytics," *International Journal of Computer Science Trends and Technology*, vol. 7, no. 2, pp. 45–52, 2019.
- [3] A. Aggarwal and R. Kaur, "Designing and Implementing Efficient NoSQL Database Systems using MongoDB," *International Journal of Computer Applications*, vol. 178, no. 20, pp. 39–45, 2019.
- [4] S. Rani and P. Kumar, "Analysis of Learning Management Systems and their Database Requirements," *International Journal of Computer Science and Information Technologies*, vol. 6, no. 4, pp. 754–759, 2013.
- [5] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2020.
- [6] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Pearson Education, 2015.
- [7] J. Fowler, "Understanding Document Databases for Educational Applications," *Journal of Information Systems and Technology*, vol. 12, no. 3, pp. 22–29, 2021.
- [8] M. T. O'Neill, "Educational Data Tracking and Student Performance Analytics," *International Journal of Emerging Technologies in Learning*, vol. 15, no. 8, pp. 75–88, 2020.
- [9] A. S. Reddy, "Evaluation of E-learning Systems Using Database-driven Approaches," *International Research Journal of Engineering and Technology*, vol. 5, no. 6, pp. 1120–1126, 2018.

